R2RML: A Step-by-step Tutorial

Christophe Debruyne, Kris McGlinn
ADAPT @ Trinity College Dublin

Introduction

The goal of this tutorial is to familiarize the reader with some core concepts of R2RML, and is adapted from the version online here[1]. The R2RML processor we will use is R2RML-F[2], though we will not avail of any functionality outside of R2RML's specification. We will assume that the reader has downloaded or installed an R2RML processor, as per the set-up instructions sent out before this tutorial. If you have not already downloaded the processor, please jump to the Appendix "setting up and testing the R2RML processor".

Specifically, we are looking at data from the different Vasculitis registries within the context of the FAIRVASC project. We will therefore focus on the mappings developed for the RKD registry in Ireland (we also make sample R2RML mappings available for each of the registries).

While R2RML was intended for relational databases, R2RML-F allows one to access CSV files as relational tables. This tutorial will use this feature so that a relational database will not be required for this tutorial. We use the H2 Database Engine to access CSV files as tables. This means that column names are capitalized (i.e., patient becomes PATIENT).

The CSV file we will transform into RDF is RKD.csv (which is also contained in the zip file you have downloaded). This file contains information on simulated vasculitis patients in the Republic of Ireland; their Date of Birth, diagnosis, etc.. The CSV file contains the following rows which are relevant to this tutorial. There are three classes covered, as defined in our ontology. These are; "Patient", "Diagnosis" and "ANCA". Patients have at most one value for year of birth, gender, date of death and cause of death, as well as one value for ANCA IF and ANCA Specificity. A patient may have multiple diagnosis though, and so we use the data of diagnosis combined with their record number to uniquely identify these.

Table 1 Patient: ID, Year of Birth, Gender, Date of Death, Cause of Death, Diagnosis, ANCA: Immunofluorescence and ANCA Specificity

RECORD_ID	YEAR_OF_BIRTH	GENDER	DATE_OF_EVENT	CAUSE_OF_DEATH_KNOWN	DATE_OF_DIAGNOSIS	SMALL_VESSEL_VAS_ANCA	ANCA_IF	ANCA_SPEC
2134	1965	2			31/09/2011	4	1	1
7865	1944	1			02/03/2011	3	1	1
46445	1957	1	13/11/2019		22/06/2012		4	2
34534	1942	2	06/07/2011	PE	08/11/2008	4	2	1

Before we start, we will first create the configuration file for our R2RML-F processor. The contents of that file, which we call config.properties is as follows:

[1] https://github.com/chrdebru/r2rml-tutorial
[2] https://github.com/chrdebru/r2rml

```
CSVFiles = RKD.csv
mappingFile = fairvasc_rkd_test_mapping.ttl
outputFile = fairvasc_rkd_test.ttl
format = TURTLE
```

This informs the processor that RKD.csv will be transformed into RDF using the mapping provided by fairvasc_rkd_test_mapping.ttl. The RDF will be formatted into TURTLE and written to fairvasc_rkd_test.ttl

We now create fairvasc_rkd_test_mapping.ttl and add the following prefixes:

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix fvc: <http://ontologies.adaptcentre.ie/fairvasc#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

The names spaces rr, rdfs, and xsd refer to the namespaces for R2RML, RDFS, and XSD respectively. We furthermore assume the existence of an ontology prefixed with fvc for some concepts and relations.

A Triples Map for RARE KIDNEY DISEASE PATIENTS

We first create the triples map for our patient's class. Every triples map needs a logical table and a subject map. The subject map is responsible for creating the subjects and any type declarations.

```
# PREFIXES APPEAR HERE
<#patient>

          rr:logicalTable [ rr:tableName "RKD" ];

          rr:subjectMap [
                    rr:template "http://data.fairvasc.ie/resource/patient/{RECORD_ID}" ;
                    rr:class fvc:Patient;
          ] ;

.
```

For every row in the table, we use RECORD_ID combined with a base URI http://data.fairvasc.ie/resource/patient/ to create the URI of the subject. We also declare that each subject is an instance of fvc:Patient.

We now execute the mapping by clicking on "run.bat" or alternatively typing the command "java -jar r2rml/r2rml.jar config.properties"

The following triples should be generated:

```
<http://data.fairvasc.ie/resource/record/encounter/patient/46445>
        a       <http://ontologies.adaptcentre.ie/fairvasc#Patient>.

<http://data.fairvasc.ie/resource/record/encounter/patient/2134>
        a       <http://ontologies.adaptcentre.ie/fairvasc#Patient>.

<http://data.fairvasc.ie/resource/record/encounter/patient/34534>
        a       <http://ontologies.adaptcentre.ie/fairvasc#Patient>.

<http://data.fairvasc.ie/resource/record/encounter/patient/7865>
        a       <http://ontologies.adaptcentre.ie/fairvasc#Patient>.
```

Now let's extend the mappings to include some data about each Patient, i.e. a patient ID, year of birth, gender, date of death, and cause of death. We do this by defining rr:predictaeObjectMap. Here we declare the name of the predicate, i.e. "patientID", and the object that will be mapped to,

i.e. “RECORD_ID” column in the CSV. We can also specify the rr:datatype for the object, e.g. “xsd:string”.

```
# PREFIXES APPEAR HERE
<#patient>

          rr:logicalTable [ rr:tableName "RKD" ];

          rr:subjectMap [
                    rr:template "http://data.fairvasc.ie/resource/patient/{RECORD_ID}" ;
                    rr:class fvc:Patient;
          ] ;

          rr:predicateObjectMap [
                    rr:predicate fvc:patientID ;
                    rr:objectMap [
                              rr:column "RECORD_ID" ;
                              rr:datatype xsd:string;
                    ] ;
          ] ;


          rr:predicateObjectMap [
                    rr:predicate fvc:yearOfBirth ;
                    rr:objectMap [
                              rr:column "YEAR_OF_BIRTH" ;
                              rr:datatype xsd:gYear;
                    ] ;
          ] ;

          # create a predicate object map for GENDER, the datatype is xsd:string
          # create a predicate object map for DATE_OF_EVENT, the datatype is xsd:string
          # NOTE: DATE_OF_EVENT is not correctly formatted yet, that will be covered later on

          rr:predicateObjectMap [
                    rr:predicate fvc:causeOfDeath ;
                    rr:objectMap [
                              rr:column "CAUSE_OF_DEATH_KNOWN" ;
                              rr:datatype xsd:string;
                    ] ;
          ];
.
```

We now execute the mapping by clicking on “run.bat” or alternatively typing the command “java -jar r2rml/r2rml.jar config.properties”

The following triples should be generated:

```
<http://data.fairvasc.ie/resource/patient/7865>
        a       <http://ontologies.adaptcentre.ie/fairvasc#Patient> ;
        <http://ontologies.adaptcentre.ie/fairvasc#patientID>
                "7865" ;
        <http://ontologies.adaptcentre.ie/fairvasc#yearOfBirth>
                "1944"^^<http://www.w3.org/2001/XMLSchema#gYear> .

<http://data.fairvasc.ie/resource/patient/34534>
        a       <http://ontologies.adaptcentre.ie/fairvasc#Patient> ;
        <http://ontologies.adaptcentre.ie/fairvasc#causeOfDeath>
                "PE" ;
        <http://ontologies.adaptcentre.ie/fairvasc#patientID>
                "34534" ;
        <http://ontologies.adaptcentre.ie/fairvasc#yearOfBirth>
                "1942"^^<http://www.w3.org/2001/XMLSchema#gYear> .

<http://data.fairvasc.ie/resource/patient/46445>
        a       <http://ontologies.adaptcentre.ie/fairvasc#Patient> ;
        <http://ontologies.adaptcentre.ie/fairvasc#patientID>
                "46445" ;
        <http://ontologies.adaptcentre.ie/fairvasc#yearOfBirth>
                "1957"^^<http://www.w3.org/2001/XMLSchema#gYear> .

<http://data.fairvasc.ie/resource/patient/2134>
```

```
        a       <http://ontologies.adaptcentre.ie/fairvasc#Patient> ;
        <http://ontologies.adaptcentre.ie/fairvasc#patientID>
                "2134" ;
        <http://ontologies.adaptcentre.ie/fairvasc#yearOfBirth>
                "1965"^^<http://www.w3.org/2001/XMLSchema#gYear> .
```

Here we see that triples are generated only for those triples which have a corresponding value in the CSV. So, for patient 2134 and patient 7856, there is no date of death, or cause of death and for patient 46445 there is no cause of death, but a date of death is specified.

You may also notice that fvc:gender is recorded as an integer value, e.g. "1". Let us do some simple transformations so that we match our agreed values for the data dictionaries. We use an SQL so that this is now represented as a string "Male" when 1 and string "Female" when 2 and "Undetermined" when 3.

To do this, in the mapping we change the text for our rr:logicalTable to:

```
rr:logicalTable [ rr:sqlQuery """SELECT
          RECORD_ID,
          YEAR_OF_BIRTH,
          CONCAT(FORMATDATETIME(PARSEDATETIME(DATE_OF_EVENT, 'dd/MM/yyyy'),
'yyyy-MM-dd'),'T00:00:00') AS DEATH_DATE,
          CAUSE_OF_DEATH_KNOWN,
          GENDER,
                    CASE
                    WHEN GENDER = 1 THEN 'Male'
                    WHEN GENDER = 2 THEN 'Female'
                    WHEN GENDER = 3 THEN 'Undetermined'
                    END AS GENDER_
          FROM RKD""" ];
```

This selects out gender, and replaces the value. This simple transformation will be useful for data sets which use integer values to represent enumerated strings, and for mapping values to our agreed values. We also formatted the values of "DATE_OF_EVENT" to comply with the xsd:dateTime representation. We formatted the date to transform 08/08/1984 into 1984-08-08, and added a time to it. The resulting value has been assigned to the variable DEATH_EVENT. Now change the predicate object map to use DEATH_EVENT instead of DATE_OF_EVENT.

Save and run the "run.bat" file again. Now you will see that the gender is represented as a string.

GENERATING TRIPLES FOR ANCA & DIAGNOSIS

Each patient has ANCA related lab tests and also one or more diagnosis. Here we show how to relate a patient to these values. First, we must create triplesmaps for the two classes we wish to represent. Let us do diagnosis first. For Diagnosis, each patient may have one or more entries, and so we use both the RECORD_ID and the DATE_OF_DIAGNOSIS to form the subject URI. We again use SQL CASE to select between the possible values for mainDiagnosis. We also format the date to adhere to xsd dateTime for dateOfDiagnosis.

```
<#diagnosis>
rr:logicalTable [ rr:sqlQuery """SELECT
          RECORD_ID,
          FORMATDATETIME(PARSEDATETIME(DATE_OF_DIAGNOSIS, 'dd/MM/yyyy'),
'yyyy-MM-dd')||'T00:00:00' AS DIAGNOSIS_DATE,
                    CASE
                    WHEN SMALL_VESSEL_VAS_ANCA = 1 THEN 'ANCA vasculitis unclassified -
ORPHA:156152'
                    WHEN SMALL_VESSEL_VAS_ANCA = 2 THEN 'Eosinophilic granulomatosis with
polyangiitis (EGPA)'
                    WHEN SMALL_VESSEL_VAS_ANCA = 3 THEN 'Granulomatosis with polyangiitis
(GPA)'
                    WHEN SMALL_VESSEL_VAS_ANCA = 4 THEN 'Microscopic polyangiitis (MPA)'
```

```
                    END AS DIAGNOSIS
FROM RKD""" ];

rr:subjectMap [
          rr:template
"http://data.fairvasc.ie/resource/rkd/diagnosis/{RECORD_ID}{DIAGNOSIS_DATE}" ;
          rr:class fvc:Diagnosis;
] ;

rr:predicateObjectMap [
          rr:predicate fvc:mainDiagnosis ;
          rr:objectMap [
                    rr:column "DIAGNOSIS" ;
                    rr:datatype xsd:string;
          ] ;
] ;

rr:predicateObjectMap [
          rr:predicate fvc:dateOfDiagnosis ;
          rr:objectMap [
                    rr:column "DIAGNOSIS_DATE" ;
                    rr:datatype xsd:dateTime;
                    ] ;
          ] ;

.
```

Now let's repeat for ANCA.

```
<#anca>
rr:logicalTable [ rr:sqlQuery """SELECT
          RECORD_ID,
          CASE
                    WHEN ANCA_SPEC = 1 THEN 'MPO'
                    WHEN ANCA_SPEC = 2 THEN 'ELISA negative'
                    WHEN ANCA_SPEC = 3 THEN 'PR3'
                    WHEN ANCA_SPEC = 4 THEN 'Other'
                    WHEN ANCA_SPEC = 5 THEN 'No ELISA performed'
                    WHEN ANCA_SPEC = 5 THEN 'MPO and PR3'
                    END AS ANCASPEC,
          CASE
                    WHEN ANCA_IF = 1 THEN 'C'
                    WHEN ANCA_IF = 2 THEN 'P'
                    WHEN ANCA_IF = 3 THEN 'Atypical'
                    WHEN ANCA_IF = 4 THEN 'Negative'
                    WHEN ANCA_IF = 5 THEN 'Not tested'
                    END AS ANCAIF
          FROM RKD"""];

          rr:subjectMap [
                    rr:template "http://data.fairvasc.ie/resource/rkd/anca/{RECORD_ID}" ;
                    rr:class fvc:ANCA;
          ] ;

          rr:predicateObjectMap [
                    rr:predicate fvc:ancaIF;
                    rr:objectMap [
                              rr:column "ANCAIF" ;
                              rr:datatype xsd:string;
                    ] ;
          ] ;

          rr:predicateObjectMap [
                    rr:predicate fvc:ancaSpec ;
                    rr:objectMap [
                              rr:column "ANCASPEC" ;
                              rr:datatype xsd:string;
                    ] ;
          ] ;
.
```

TYING IT ALL TOGETHER

Now that we have a triples map for Diagnosis and Anca, we can create a predicate object map that relates patients to both of these. It is true that in this simple case we can avail of a "simple" predicate object map, but we will now illustrate a predicate object map referring to another triples map with join conditions. The R2RML engine will thus create triples based on an SQL join. We extend the first triples map as follows:

```
        rr:predicateObjectMap [
                rr:predicate fvc:hasANCA ;
                rr:objectMap [
                        rr:parentTriplesMap <#anca> ;
                        rr:joinCondition [
                                rr:child "RECORD_ID" ;
                                rr:parent "RECORD_ID" ;
                        ] ;
                ];
        ] ;

        rr:predicateObjectMap [
                rr:predicate fvc:hasDiagnosis ;
                rr:objectMap [
                        rr:parentTriplesMap <#diagnosis> ;
                        rr:joinCondition [
                                rr:child "RECORD_ID" ;
                                rr:parent "RECORD_ID" ;
                        ] ;
                ];
        ] ;
```

We now have an object relation for Patient called “hasANCA” which points to a subject of type ANCA, which in turn has two data relations ancaIF and ancaSPEC. We also have an object relation between Patient and Diagnosis “hasDiagnosis”.

Let us run the R2RML processor again, and you should see data that resembles below:

```
<http://data.fairvasc.ie/resource/rkd/patient/34534>
        a       <http://ontologies.adaptcentre.ie/fairvasc#Patient> ;
        <http://ontologies.adaptcentre.ie/fairvasc#causeOfDeath>
                "PE" ;
        <http://ontologies.adaptcentre.ie/fairvasc#dateOfDeath>
                "2011-07-06T00:00:00"^^<http://www.w3.org/2001/XMLSchema#dateTime> ;
        <http://ontologies.adaptcentre.ie/fairvasc#gender>
                "Female" ;
        <http://ontologies.adaptcentre.ie/fairvasc#hasANCA>
                <http://data.fairvasc.ie/resource/rkd/anca/34534> ;
        <http://ontologies.adaptcentre.ie/fairvasc#hasDiagnosis>
                <http://data.fairvasc.ie/resource/rkd/diagnosis/345342008-11-08T00%3A00%3A00>
;
        <http://ontologies.adaptcentre.ie/fairvasc#patientID>
                "34534" ;
        <http://ontologies.adaptcentre.ie/fairvasc#yearOfBirth>
                "1942"^^<http://www.w3.org/2001/XMLSchema#gYear> .

<http://data.fairvasc.ie/resource/rkd/diagnosis/345342008-11-08T00%3A00%3A00>
        a       <http://ontologies.adaptcentre.ie/fairvasc#Diagnosis> ;
        <http://ontologies.adaptcentre.ie/fairvasc#dateOfDiagnosis>
                "2008-11-08T00:00:00"^^<http://www.w3.org/2001/XMLSchema#dateTime> ;
        <http://ontologies.adaptcentre.ie/fairvasc#mainDiagnosis>
                "Microscopic polyangiitis (MPA)" .

<http://data.fairvasc.ie/resource/rkd/anca/34534>
        a       <http://ontologies.adaptcentre.ie/fairvasc#ANCA> ;
        <http://ontologies.adaptcentre.ie/fairvasc#ancaIF>
                "P" ;
        <http://ontologies.adaptcentre.ie/fairvasc#ancaSpec>
                "MPO" .
```

Additional Exercise

Now look at the simulated data from your own pilot's registry. See if you can adjust the R2RML code to generate the same RDF for your own registry. If you have any trouble, please consult the R2RML mappings that are already done and stored in folder: R2RML/

Here - https://drive.google.com/drive/u/1/folders/1llkSy9qS-r5YNcII6ii6WgRokGYp13Ma

RESOURCES

Here you will find again the links to all the pre-requisite material for this training session.

Data, Data Dictionaries and Ontology

Simulated patient data was generated for each pilot site in FAIRVASC, i.e. for; Ireland, Germany, Sweden, Czech Republic, Poland and France. This simulated data is available here:

https://drive.google.com/drive/folders/1kTyF76lOwExudmTLq9D2jswoeEDFy_16?usp=sharing

We also make each data dictionary available here: R2RML /Pilot Data Dictionaries/

We make the FAIRVASC ontology available here: R2RML/fairvasc.owl

Mappings and Generated RDF

The R2RML mappings for each site are also made available here R2RML /Pilot Simulated Data/

And the generated RDF here R2RML /Pilot Simulated Data/

Appendix: Setting up and testing the R2RML Processor

In this tutorial, we are simply going to use the R2RML jar[3] file to convert your CSV data according to the defined R2RML file. You will be downloading the JAR file, along with its dependencies. You will also need to download the configuration file and a batch file. These are all zipped together in a file called "r2rml-f.zip located here –R2RML/

It is assumed you have Java installed. You run the R2RML processor by clicking the batch file called "run.bat", or alternatively you can use the following command:

java -Xmx1024m -jar r2rml.jar config.properties

For more on downloading and installing R2RML-F, follow this link - https://github.com/chrdebru/r2rml

Or view this tutorial:

- macOS: https://www.youtube.com/watch?v=Qp_VGXAH_T4&feature=youtu.be
- Windows: https://www.youtube.com/watch?v=LPB_zpXX_b8&feature=youtu.be

The tutorials start from the folder which you can download here:

Training Session 2 / r2rml-starting-point

[3] A JAR (Java ARchive) is a package file format typically used to aggregate many Java class files and associated metadata and resources (text, images, etc.) into one file for distribution.